

Software Engineering Final Project



Team Members

Farah Walid (23010036)

Basmala Hossam El-Din (23011052)

Nour Essam (23010035)

Mariam Ahmed (23012058)

Rodina Mohamed (23010014)

Mesk Ayman (23011536)

System Overview:

- **Project Title:** Kenzee Locksmith Digital Service & Dispatch System
- **Problem Description:** Currently, Kenzee Locksmith operates through manual processes. Traditional phone-based bookings lead to several inefficiencies:
 - **Response Latency:** Emergency lockouts require immediate assistance, but manual dispatching causes delays.
 - **Lack of Real-time Tracking:** Customers have no visibility into a technician's ETA.
 - **Manual Record Keeping:** Booking data and service history are not centralized, making business scaling difficult.

System Objectives:

- **Automation:** To automate the intake of service requests and technician assignment.
- **Transparency:** To provide users with digital service quotes and estimated arrival times.
- **Reliability:** To ensure (24/7) accessibility for emergency locksmith services through a web interface.
- **Centralization:** To create a digital record of all transactions, service types, and customer feedback.

Stakeholders:

- **Customers:** Users in need of residential, commercial, or emergency automotive locksmith services.
- **Technicians:** Field employees who receive job details and update status via the system.
- **Admin (Business Owner):** Manages staff scheduling, reviews financial reports, and monitors service quality.
- **Third-Party Providers:** Payment processors (Stripe) and Mapping APIs (Google Maps) for technician tracking.

Work Breakdown Structure (WBS):

1. Kenzee Locksmith Digital System

1.1. Requirements Engineering:

- 1.1.1. Elicitation & Stakeholder Analysis
- 1.1.2. Functional & Non-Functional Specification
- 1.1.3. Requirements Analysis

1.2. System Modeling (UML):

- 1.2.1. Use Case Diagram
- 1.2.2. Context Diagram
- 1.2.3. Activity Diagram
- 1.2.4. State Machine Diagram

1.3. Project Management:

- 1.3.1. GANTT Chart
- 1.3.2. PERT Diagram

1.4. Implementation:

- 1.4.1. Web Interface Development (HTML/CSS/JS)

Functional Requirements:

1. Service Category Selection

- User Requirement: Customers must be able to select between Residential, Commercial, and Automotive services.
- System Requirement: The system shall render interactive service cards that display unique content based on the selected category.

2. Location Awareness

- User Requirement: Users must be able to identify the service area.
- System Requirement: The system shall display a map to verify serviceability within a specific geographic range.

3. Interactive User Interface

- Users must be able to navigate the site sections (Home, About, Services, Contact) using a menu.
- System Requirement: The system shall implement a sticky navigation bar with anchor links to specific page sections.

4. Emergency Contact Integration

- User Requirement: Users in a rush must be able to trigger a phone call or email directly from the "Contact Us" section.

- System Requirement: The system shall utilize tel: and mailto: protocols to launch external communication apps on click.

5. Booking Information Intake

- User Requirement: Users must be able to fill out a contact form with their name, phone number, and service details.
- System Requirement: The system shall capture user input via HTML5 forms and perform client-side validation for required fields.

6. Responsive Page

- User Requirement: Users must be able to access all dispatch features clearly on a smartphone or tablet.
- System Requirement: The system shall utilize CSS Media Queries to ensure responsive layout across mobile and desktop.

7. User Authentication

- User Requirement : Users must be able to create a personal account and log in securely to track their requests.
- System Requirement: The system shall verify user credentials against a hashed database and manage active sessions via secure tokens.

8. Real-Time Dispatching

- User Requirement : The system must automatically notify a technician when a new service request is submitted in their area.
- System Requirement: The system shall trigger a POST request to the Technician Dashboard API whenever a new record is added to the Requests table.

9. Automated SMS Alerts

- User Requirement : Users should receive text updates so they don't have to keep the web browser open to know the technician's status.
- System Requirement: The system shall integrate with a notification API to send automated SMS triggers when the Status changes to "pending" or "Arrived".

10. Rating system

- User Requirement : Customers must be able to rate the technician and leave a written review once the service is finalized.
- System Requirement: The system shall store numerical ratings and text strings in the Reviews table, linked to both the User_ID and Technician_ID.

Non-Functional Requirements:

1. Performance

- User Requirement: Users expect the system to load quickly and respond without delays during use.
- System Requirement: The system shall load pages within ≤ 3 seconds and process API requests within ≤ 2 seconds under normal conditions.

2. Availability

- User Requirement: Users must be able to access the service at any time, especially in emergencies.
- System Requirement: The system shall operate 24/7 with at least 99% uptime, excluding scheduled maintenance.

3. Usability

- User Requirement: Users must be able to use the system easily without technical knowledge.
- System Requirement: The system shall provide an intuitive interface allowing users to complete booking in ≤ 3 steps with clear navigation.

4. Security

- User Requirement: Users must feel that their personal data is safe and protected.
- System Requirement: The system shall use HTTPS encryption, store passwords as hashed values, and protect against common attacks (SQL Injection, XSS).

5. Scalability

- User Requirement: Users expect the system to remain efficient even as the number of users increases.
- System Requirement: The system shall support increasing numbers of users, technicians, and requests without performance degradation.

6. Reliability

- User Requirement: Users must receive accurate service information without errors or system crashes.
- System Requirement: The system shall ensure consistent operation, accurate data processing, and automatic retry of failed operations.

7. Maintainability

- User Requirement: The system should be improved and updated without disrupting user experience.

- **System Requirement:** The system shall be modular and well-documented to allow easy updates and maintenance.

8. Compatibility

- **User Requirement:** Users must be able to access the system from different browsers and devices.
- **System Requirement:** The system shall support modern web browsers (Chrome, Edge, Firefox, Safari) and multiple device types.

9. Data Integrity

- **User Requirement:** Users expect their data (bookings, reviews) to be accurate and stored correctly
- **System Requirement:** The system shall validate inputs and ensure consistency and correctness of stored data.

10. Interoperability

- **User Requirement:** Users expect external services such as maps, payments, and notifications to work seamlessly.
- **System Requirement:** The system shall integrate with third-party APIs (SMS, payment, maps) using standard protocols (REST/JSON).

Requirements Analysis:

Ambiguities: Ambiguities are requirements that are unclear or open to multiple interpretations.

- **Performance:** The phrase "respond without delays" is vague from the user perspective and depends on network conditions and system load.
- **Availability:** The system is required to operate (24/7) with 99% uptime, but it is unclear how downtime or failures will be handled.
- **Usability:** Terms like "easy to use" and "intuitive interface" are subjective and not measurable.
- **Security:** Security is partially defined (HTTPS, hashing), but advanced measures (multi-factor authentication, data protection levels) are not specified.
- **Scalability:** The requirement does not define the expected number of users or system limits.
- **Reliability:** "Accurate service information" and "automatic retry" lack clear measurable criteria (retry attempts, acceptable error rates).
- **Maintainability:** "Well-documented" is unclear without specifying documentation standards or formats.

- **Compatibility:** "Modern browsers" is not specific (which versions are supported).
- **Data Integrity:** The requirement does not define backup strategies or recovery mechanisms.
- **Interoperability:** Integration with third-party APIs is mentioned, but no fallback behavior is defined if those services fail.

Conflicts: Conflicts are situations where requirements contradict each other or create trade-offs.

- **Performance vs Security:** Implementing strong security measures (encryption, validation) may reduce system speed.
- **Availability vs Maintainability:** The system must be available (24/7), but maintenance and updates may require downtime.
- **Usability vs Security:** Making the system simple to use may reduce the level of security (fewer authentication steps).
- **Performance vs Scalability:** Supporting a large number of users may affect system performance and response time.
- **Reliability vs Interoperability:** Dependence on third-party services (SMS, maps) may affect system reliability if those services fail.
- **Compatibility vs Design:** Supporting many devices and browsers may limit the use of advanced design features.
- **Data Integrity vs Performance:** Strong validation and consistency checks may slow down system operations.

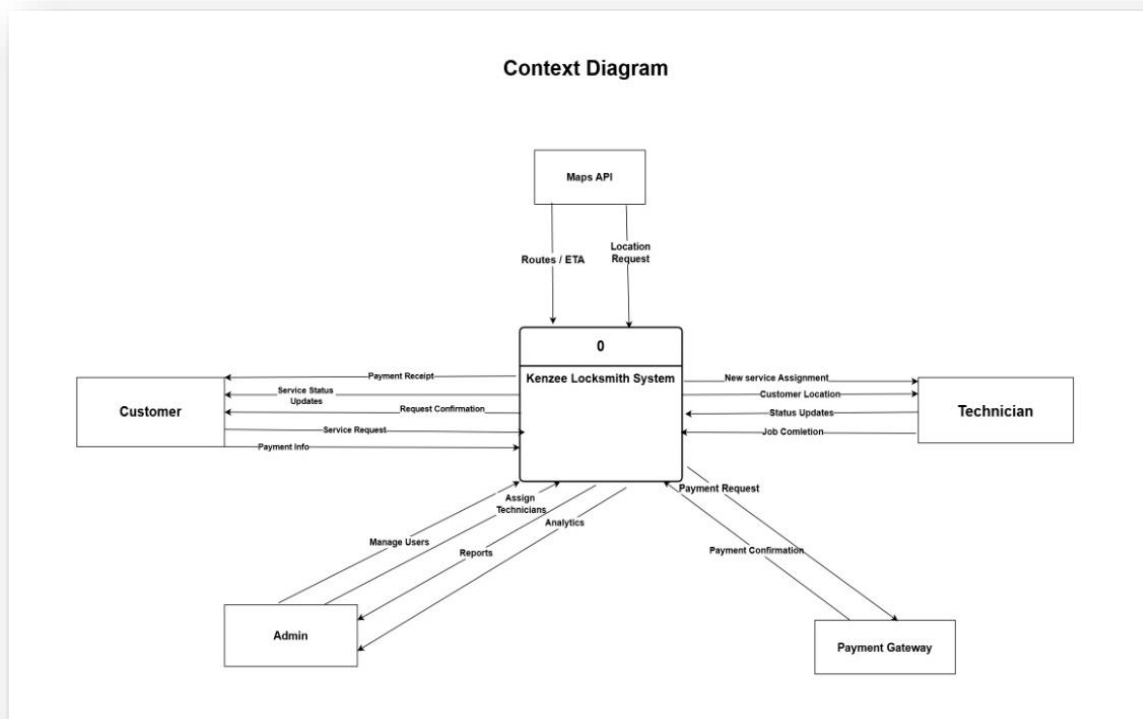
Context Diagram

The Context Diagram represents the Kenzee Locksmith Digital Service & Dispatch System as a single process (Level 0), showing how it interacts with external entities. The system is positioned at the center, with surrounding actors connected to it through data flows, illustrating how information moves to and from the system in a clear and structured manner.

♦ Customer Interaction

- Submit service requests.
- Provide payment details.
- Receive:
 - Request confirmation.
 - Service status updates.
 - Payment receipts.

- Enables customers to request, track, and pay for services easily.
- ♦ **Technician Interaction**
 - Receive service assignments with location and customer details.
 - Send:
 - Status updates.
 - Job completion confirmation.
 - Supports real-time dispatching and efficient service delivery.
- ♦ **Admin Interaction**
 - Manage users.
 - Assign technicians.
 - View reports and analytics.
 - Ensures system control and monitoring.
- ♦ **External Systems Integration**
 - Map API
 - Provides routes, ETA, and location data.
 - Payment Gateway
 - Processes payments
 - Sends payment confirmations
 - Ensures smooth integration with third-party services

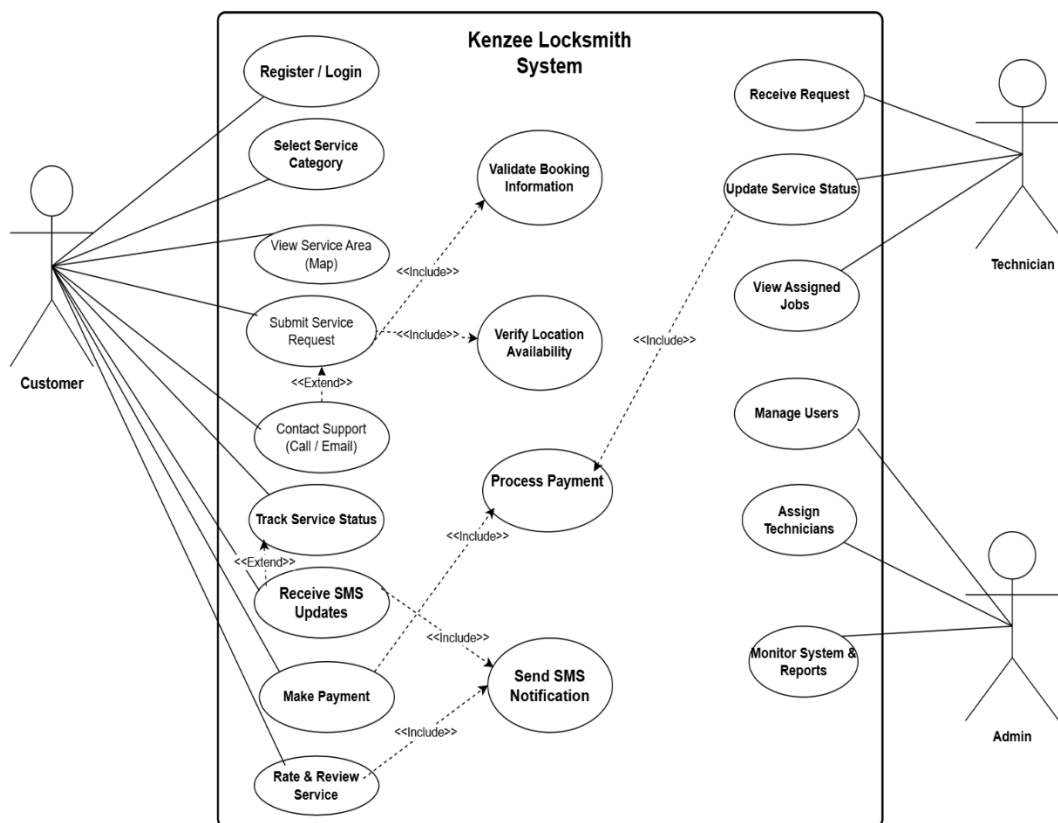


Use Case Diagram

The Use Case Diagram provides an overview of how different actors interact with the Kenzee Locksmith System. The system is represented as a central boundary that contains all its functionalities, while the external actors: Customer, Technician, and Admin are positioned outside this boundary. Their interactions with the system are illustrated through clearly defined use cases, showing how each actor performs specific actions within the system.

- ◆ Customer Interaction
 - Register and log in.
 - Select service category (Residential, Commercial, Automotive).
 - View service area on map.
 - Submit service request.
 - Track service status.
 - Make payments.
 - Receive SMS updates.
 - Rate and review service.
 - Use cases align with functional requirements.
- ◆ Technician Interaction
 - Receive service requests.
 - View assigned jobs.
 - Update service status.
 - Supports real-time dispatching and efficient service.
- ◆ Admin Interaction
 - Manage users.
 - Assign technicians.
 - Monitor system performance.
 - Generate reports.
 - Ensures system control and management.
- ◆ Business Rules Representation
 - Include Relationships (Mandatory)
 - Submit Service Request includes:
 - Validate Booking Information.
 - Verify Location Availability.
 - Make Payment includes:
 - Process Payment.

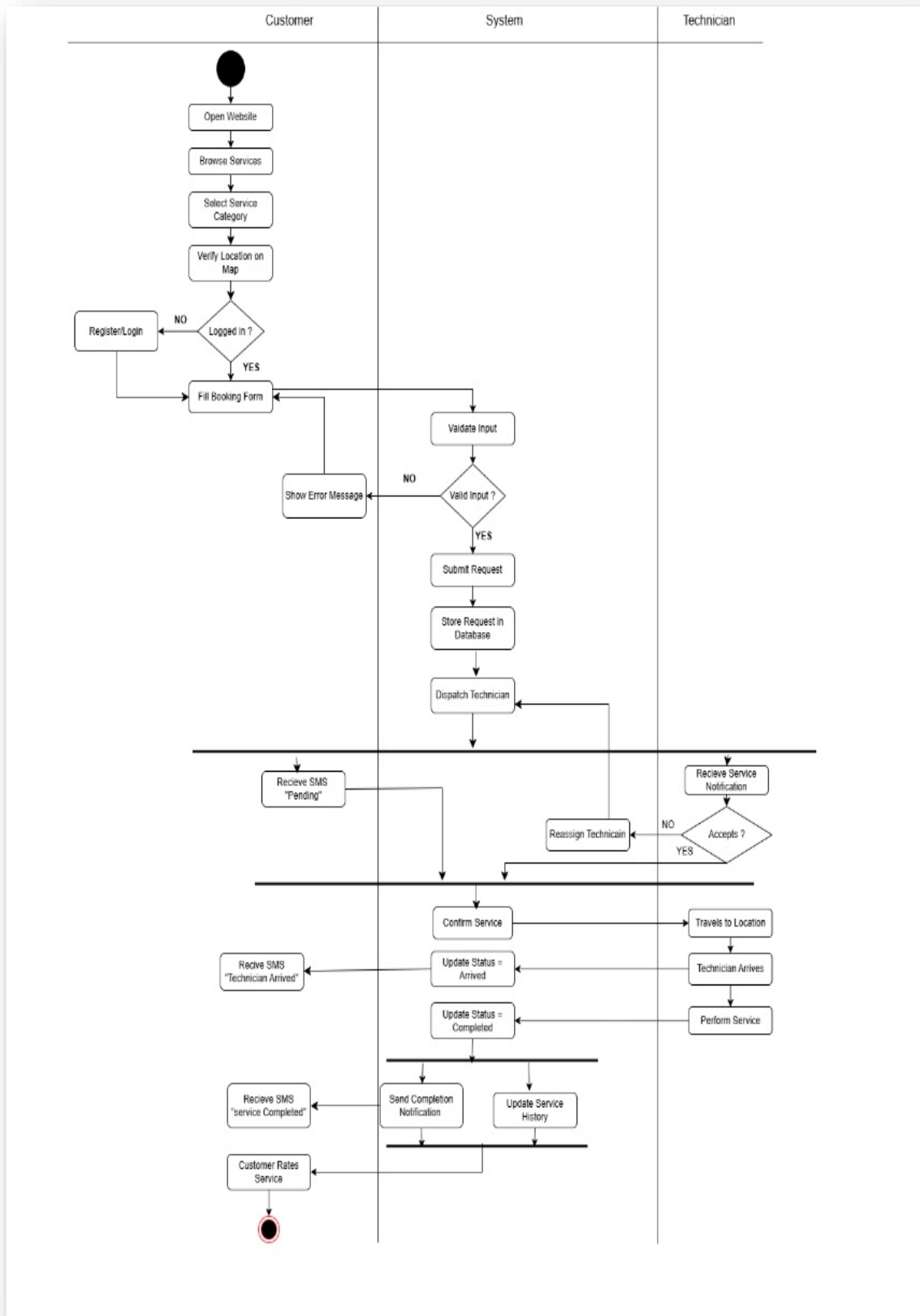
- SMS Updates include:
 - Send SMS Notification.
- Ensures required steps are always executed.
- Extend Relationships (Optional)
 - Contact Support extends Submit Service Request.
 - Receive SMS Updates extends Track Service Status.
 - Rate & Review Service happens after completion.
 - Represents optional or conditional behavior.
- ◆ Compliance with Requirements
 - Clear and readable layout.
 - Well-structured organization.
 - Fully labeled elements.
 - Consistent with functional requirements.
 - Business rules clearly shown using <<include>> and <<extend>>.



Activity Diagram

This activity diagram represents the workflow of the service booking system involving three main roles: Customer, System, and Technician.

- The process starts when the customer opens the website.
- The customer browses services, selects a category, and verifies location.
- The system checks if the user is logged in:
 - If not logged in, the user must register/login.
 - If logged in, the user proceeds directly.
- The customer fills the booking form.
- The system validates the input:
 - If input is invalid, an error message is shown.
 - If input is valid, the request is submitted.
- The system stores the request in the database and dispatches a technician.
- The customer receives a "Pending" SMS notification.
- The technician receives the request and decides:
 - If rejected, the system reassigns another technician.
 - If accepted, the service is confirmed.
- The technician travels to the location and arrives.
- The system updates the status to "Arrived" and notifies the customer.
- The technician performs the service.
- The system updates the status to "Completed" and sends a completion notification.
- The system updates service history.
- The customer receives completion SMS and rates the service.
- The process ends after rating.



GANTT Chart

The Gantt Chart is a project management tool used to represent project activities over time , it provides a visual timeline that shows when each phase starts and ends.

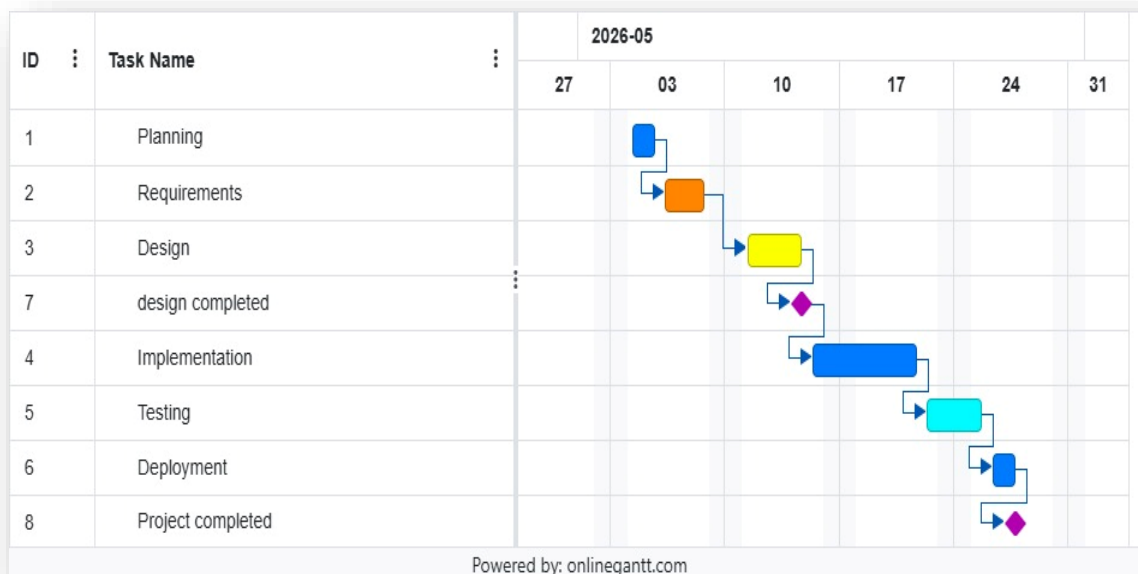
The purpose of using it is to illustrate the scheduling of the project phases based on SDLC it helps in understanding the sequence of activities , managing time efficiently and ensuring that all phases are completed within the planned timeframe.

The GANTT chart was constructed based on SDLC phases:

- Planning
- Requirements
- Design
- Implementation
- Testing
- Deployment

Each phase is assigned a specific duration and arranged sequentially to reflect the logical flow of the system development process.

The GANTT chart is presented at a high level to maintain clarity and simplicity.



PERT Diagram

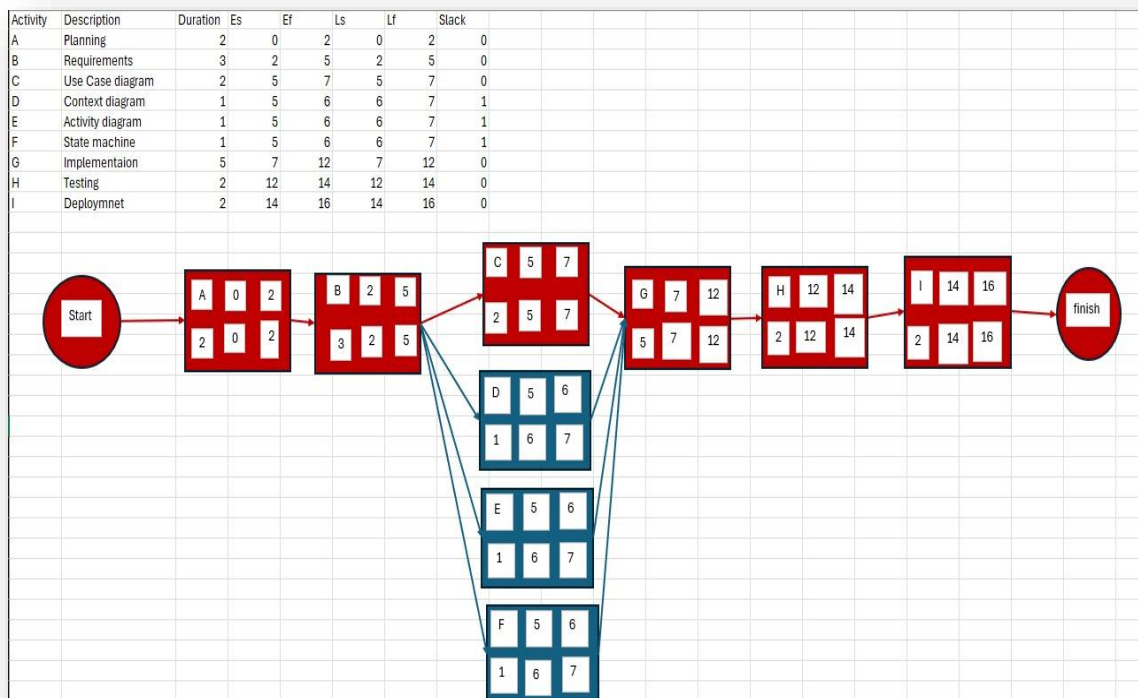
The PERT (Program Evaluation and Review Techniques) diagram is a project management tool used to represent task dependencies and the order of execution, it focuses on how activities are related rather than when they occur.

The PERT diagram was constructed based on the SDLC phases, The activities are connected according to their logical dependencies, the Design phase is decomposed into multiple parallel tasks:

- Use case diagram
- Context Diagram
- Activity Diagram
- State Machine Diagram

The critical path represents the longest sequence of dependent activities, It determines the minimum completion time of the project.

The design phase was specifically decomposed in the pert diagram because it contains multiple independent modeling activities that can be performed simultaneously, however other phases were not decomposed because they are either sequential in nature or don't include independent parallel tasks.

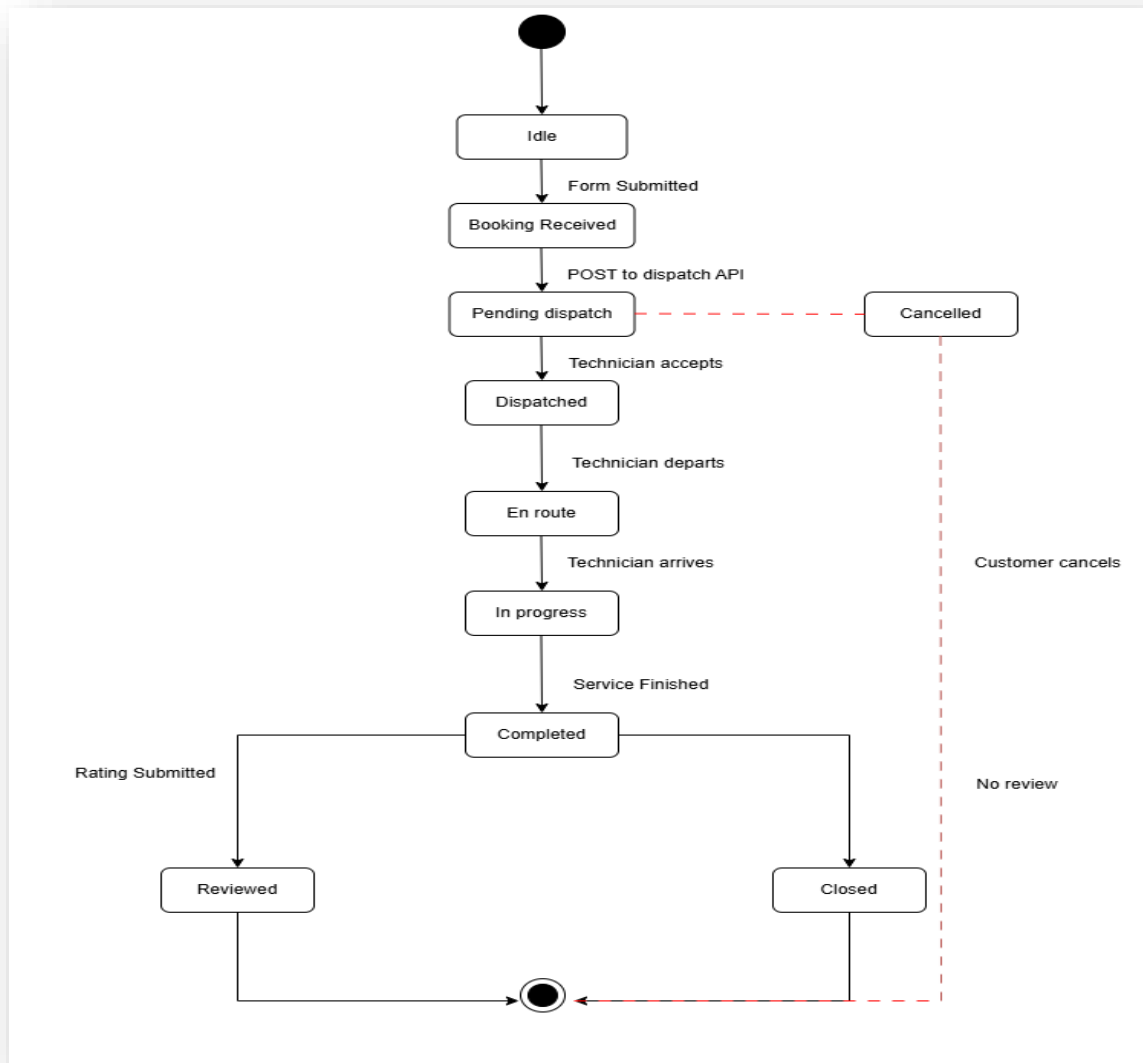


State-Machine Diagram

The state machine diagram illustrates the lifecycle of a service request in the system, showing how it moves between different states based on customer actions, system processing, and technician responses. It focuses on the status changes of a single request from creation until completion or cancellation, ensuring clear tracking and control of the service process.

- The diagram represents the lifecycle of a service request.
- The process starts at Idle, where the system waits for a request.
- When the customer submits a request → Booking Received.
- The system processes the request → Pending Dispatch.
- If a technician accepts → Dispatched.
- Technician starts traveling → En Route.
- Technician begins work → In Progress.
- Service is finished → Completed.
- After completion:
 - If the customer gives feedback → Reviewed.
 - If no feedback → Closed.
- Alternative flow:
 - The request can be Cancelled before technician assignment.
 - The process ends early without service completion.
- The diagram shows both normal flow and exception (cancellation) while tracking the request status from start to end.





Website Demo:

- <https://meskay.github.io/Web-Projects/>

